

CSE233 Week 6: Testing and Test Driven Development

This week, we'll discuss software testing and the test driven development (TDD) approach to building software. If code is not tested, you should assume it does not work. When looking at real software systems, you will notice that a large portion (many times a majority) of its code is purely dedicated to testing. Here, we are going to discuss unit tests, which are tests that verify that small units of software (such as an individual function) work as intended.

Note: there are a lot of repetitive examples that make this look longer than it is. Testing is a very repetitive process

The Assert Function

The `assert()` function from the `<cassert>` library allows you to assert that a certain condition is true. If the assertion fails, i.e., the condition is false, `assert` terminates the program. Below is a simple C++ program that demonstrates `assert`.

```
#include <iostream>
#include <cassert>

int main() {
    int x = 5;
    std::cout << "Beginning tests...\n";
    assert(x == 5);
    assert(x < 10);
    assert(x % 2 == 1);
    std::cout << "All tests passed.\n";
    return 0;
}
```

Test Driven Development

Test Driven Development (TDD) is a programming approach that follows the following steps: 1. Write a unit test (this test will probably fail) 2. Write the minimal amount of code to allow the test to pass 3. Refactor code 4. Repeat

In practice, I find that pure TDD is pretty cumbersome. Before streamlining the process, let's look at a small example of pure TDD in practice for demonstration purposes. Note: I am probably still skipping steps compared to a *true* TDD approach.

Here, we are defining a function called `sumDigits()`. It will take a single integer as a parameter, and compute the sum of its digits. For example, `sumDigits(1234)` will return the value 10, since $1 + 2 + 3 + 4 = 10$. First, we'll define `sumDigits()` and put a test in `main()`. Since `sumDigits()` has to return a value, we'll just have it return 0 for now.

```
#include <cassert>
#include <iostream>

int sumDigits(int digits);

int main() {
    assert(sumDigits(1) == 1);

    return 0;
}

int sumDigits(int digits) {
    return 0;
}
```

Since `sumDigits(1)` returns 0, and 0 does not equal 1, the test fails. We need to modify `sumDigits()` so that it returns the correct result. Remember, in pure TDD we do the simplest thing possible to make our test pass, even if it's stupid:

```
int sumDigits(int digits) {
    return 1;
}
```

The test will now pass, since `sumDigits(1)` returns 1, and 1 equals 1.

Now, let's add another test:

```
int main() {  
    assert(sumDigits(1) == 1);  
    assert(sumDigits(2) == 2);  
  
    return 0;  
}
```

Now, `sumDigits(2)` returns 1, and 1 does not equal 2, so the test fails, as expected. Let's modify `sumDigits()` so the test passes:

```
int sumDigits(int digits) {  
    if (digits == 2) return 2;  
    else return 1;  
}
```

The test for `sumDigits(2) == 2` will now pass. Next, we will add another test:

```
int main() {  
    assert(sumDigits(1) == 1);  
    assert(sumDigits(2) == 2);  
    assert(sumDigits(3) == 3);  
  
    return 0;  
}
```

`sumDigits(3)` will return 1, which is not 3, so the test will fail. Once again, we edit `sumDigits()` to make the test pass:

```
int sumDigits(int digits) {  
    if (digits == 3) return 3;  
    else if (digits == 2) return 2;  
    else return 1;  
}
```

Now, `sumDigits(3)` returns 3, so the test passes. Now, we can notice that so far all we are doing is returning the value of the parameter `digits`. So, since all the tests are passing, we can refactor `sumDigits()` to simply return its parameter:

```
int sumDigits(int digits) {  
    return digits;  
}
```

Now, we can run our tests again. They will pass, so we can add a new test:

```
int main() {  
    assert(sumDigits(1) == 1);  
    assert(sumDigits(2) == 2);  
    assert(sumDigits(3) == 3);  
    assert(sumDigits(4) == 4);  
  
    return 0;  
}
```

This new test also passes. At this point, we can add tests for all single digits, and they will pass. To save on space, I'll skip adding them for now. Now, let's try to add a test for a two-digit number:

```
int main() {  
    // ... single digit tests  
  
    assert(sumDigits(10) == 1);  
  
    return 0;  
}
```

In this case, the digits of the number 10 are 1 and 0. $1 + 0 = 1$, so we expect the result to be 1. If we run this test however, it will fail.

`sumDigits(10)` will return 10. Once again, we make the simplest change possible to make the test pass:

```
int sumDigits(int digits) {  
    if (digits == 10) return 1;  
    return digits;  
}
```

Now, the test we just added passes. Let's add another:

```
int main() {  
    // ... single digit tests
```

```
    assert(sumDigits(10) == 1);
    assert(sumDigits(11) == 2);

    return 0;
}
```

Once again, the test fails. So, we modify the function:

```
int sumDigits(int digits) {
    if (digits == 10) return 1;
    else if (digits == 11) return 2;
    return digits;
}
```

The tests pass, so we add another test:

```
int main() {
    // ... single digit tests

    assert(sumDigits(10) == 1);
    assert(sumDigits(11) == 2);
    assert(sumDigits(12) == 3);

    return 0;
}
```

This test fails, so we modify `sumDigits()` once again:

```
int sumDigits(int digits) {
    if (digits == 10) return 1;
    else if (digits == 11) return 2;
    else if (digits == 12) return 3;
    return digits;
}
```

At this point, all tests pass. We can also observe a trend. For each number that is greater than 10, the correct return value is 1 more than the final digit. Recall the modulus (remainder) operator: `%`. `10 % 10` is 0, `11 % 10` is 1, etc. Therefore, if `digits` is greater than 10, we can add 1 to `digits % 10`, and that should give us the value we need. Since all the tests are passing, we can refactor `sumDigits()`:

```
int sumDigits(int digits) {
    if (digits >= 10) return 1 + digits % 10;
}
```

```
    return digits;
}
```

If we run all the tests, we will find that they are still passing. I will assure you that the tests will also pass for the values between 14 and 19. Now, what happens if we add a test for `sumDigits(20)`?

```
int main() {
    // ... single digit tests
    // ... 10-19 tests
    assert(sumDigits(20) == 2);

    return 0;
}
```

If we run the tests, we'll find that `sumDigits(20)` returns 1, which is not 2. So, we still have more work to do. At this point, we modify `sumDigits()` once again:

```
int sumDigits(int digits) {
    if (digits == 20) return 2;
    else if (digits >= 10) return 1 + digits % 10;
    return digits;
}
```

At this point, you hopefully get the picture of what TDD is. I'll save us some time and perform the refactoring step now. To sum the digits of a two-digit number, we need a way to calculate the first and second digit. The modulus operator allows us to calculate the second digit for any number, $20 \% 10$ is 0, $21 \% 10$ is 1, etc. To access the first digit, recall that the division of two integers in C++ returns the floor division result of dividing the numbers. In other words, it always rounds down¹. Therefore, if we sum `digits / 10` and `digits % 10`, we will get the result we want. This even works for single digits. $1 / 10$ is 0 when performing integer division, so $1 / 10 + 1 \% 10 = 0 + 1 = 1$. Therefore, we don't need the if statement any longer.

```
int sumDigits(int digits) {
    return digits / 10 + digits % 10;
}
```

This will work for numbers from 0 to 99. But what about 100:

```
int main() {  
    // ... single digit tests  
    // ... two digit tests  
    assert(sumDigits(100) == 1);  
  
    return 0;  
}
```

We have a problem here. $100 \% 10$ is 0, but $100 / 10$ is 10. This means that our test fails. Again, to save time in this document, I'll skip some TDD steps and go straight to making the code work for numbers with any number of digits. To accomplish this, we will sum the number from right to left. Look at the example of 100. $100 \% 10 = 0$, which is the last digit of 100. $100 / 10$ is 10, which is 100 without the last digit. Now, $10 \% 10 = 0$, which is the last digit of 10. $10 / 10$ is 1, which is 10 without the last digit. $1 \% 10$ is 1, and $1 / 10$ is 0. This is as much as we can divide by 10, since $0 / 10$ will always be 0. Now, if we take the three values we got from the modulus, 0, 0, and 1, and sum them, we get 1, which is the result we wanted. Does this work for other values?

- 101: $101 \% 10 = 1$, $101 / 10 = 10$. $10 \% 10 = 0$. $10 / 10 = 1$. $1 \% 10 = 1$, $1 / 10 = 0$. $1 + 0 + 1 = 2$ ✓
- 154: $154 \% 10 = 4$, $154 / 10 = 15$, $15 \% 10 = 5$, $15 / 10 = 1$. $1 \% 10 = 1$, $1 / 10 = 0$. $4 + 5 + 1 = 10$ ✓
- 213: $213 \% 10 = 3$, $213 / 10 = 21$, $21 \% 10 = 1$, $21 / 10 = 2$. $2 \% 10 = 2$, $2 / 10 = 0$. $3 + 1 + 2 = 6$ ✓
- 5463: $5463 \% 10 = 3$, $5463 / 10 = 546$. $546 \% 10 = 6$, $546 / 10 = 54$. $54 \% 10 = 4$, $54 / 10 = 5$. $5 \% 10 = 5$, $5 / 10 = 0$. $3 + 6 + 4 + 5 = 18$ ✓

This is a reasonable amount of evidence that this approach will scale to any size number. So, what is the algorithm:

```
PROCEDURE: sumDigits  
INPUT:     digits: int  
OUTPUT:    sum: int  
  
    LET sum: int = 0  
  
    WHILE digits > 0:  
        sum += digits % 10
```

```

        digits /= 10
    END WHILE

    RETURN sum
END PROCEDURE

```

We start by initializing the sum to 0. Then, as long as the value of digits is greater than 0, we add the last digit of digits (which is digits % 10) to the sum. Then, we divide digits by 10 to remove a digit from it. Once digits becomes a single digit, dividing by 10 will make it 0, ending the loop. Now, let's implement it in C++:

```

int sumDigits(int digits) {
    int sum = 0;

    while (digits > 0) {
        sum += digits % 10;
        digits /= 10;
    }

    return sum;
}

```

Now, we can add some more tests for `sumDigits()`:

```

int main() {
    assert(sumDigits(0) == 0);
    assert(sumDigits(1) == 1);
    // ...
    assert(sumDigits(9) == 9);

    assert(sumDigits(10) == 1);
    assert(sumDigits(11) == 2);
    assert(sumDigits(12) == 3);
    assert(sumDigits(19) == 10);

    assert(sumDigits(20) == 2);
    assert(sumDigits(31) == 4);
    assert(sumDigits(42) == 6);
    assert(sumDigits(53) == 8);
    assert(sumDigits(64) == 10);
    assert(sumDigits(99) == 18);

    assert(sumDigits(100) == 1);
    assert(sumDigits(101) == 2);
    assert(sumDigits(154) == 10);
    assert(sumDigits(213) == 6);
}

```



```
    assert(sumDigits(5463) == 18);  
  
    return 0;  
}
```

We could obviously add more tests. Quantifying how many tests is sufficient is an impossible question. In theory, we'd like to test our code for every possible value. However, that simply isn't possible. Software testing requires a great deal of nuance, and even the best test writers miss cases. That is why all software has bugs. It is impossible to construct a test scenario for every possible use case. However, test suites do provide evidence that your code works for some number of examples. As stated in this week's introduction, most real world codebases have more test code than implementation code. Testing is a critical component of building and maintaining software systems.

1. Technically, there is no rounding in integer division. It is computationally faster for the computer to divide integers and totally ignore the remainder. The CPU has a separate piece of hardware for dividing integers than it does for dividing floating point numbers. Since it is much faster to divide integers, C++ assumes that if both operands are integers you mean to use the integer version of division and totally ignore the remainder.

Testing as We Write Code

The method of writing tests ahead of time and only writing the minimal and simplest code possible can be redundant. However, we still want to verify that our code works. Rather than focussing on only writing code to solve very narrow subsets of our problem, we'll instead focus on writing tests that verify that our units (in this case, individual functions), work as intended. As discussed last week, we want to break our programming tasks into manageable subtasks. We can test this code using print statements as we go, then add formal tests that verify that it works later. To demonstrate, we will write a program that solves the quadratic equation. Later, we will break our logic into separate functions and write unit tests for each of them. This allows us to write code without being bogged down by formal unit testing, but still validate that the code works later on.

Recall from algebra that for an equation of the form $ax^2 + bx + c$, we can find the roots of the equation with the quadratic equation:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The roots are the values of x that make the equation equal to 0. First, we need to consider whether the equation is actually quadratic. Notice that the value " a " is multiplying " x^2 ". If $a = 0$, then the x^2 term will be eliminated (anything times 0 is 0), and the equation is no longer quadratic. In that case, we only have to solve the linear equation $bx + c = 0$, which is much simpler. If $a = 0$ and $b = 0$, then there is no x to solve for in the equation, and there are no roots.

If the equation is quadratic, there are three cases, depending on the value of " $4ac$ " and " b^2 ". Recall from algebra that you cannot take the square root of a negative number. Any number squared is positive, so b^2 will always be positive. We are subtracting $4ac$ from b^2 , so if $4ac$ is greater than b^2 , the number in the square root will be negative. Technically, this means x has 2 imaginary roots, but we are not computing complex numbers here, and will simply report that we can't compute those roots. If $b^2 == 4ac$, the result of subtracting them is 0. This will lead to us only having one root. Otherwise, the equation will have 2 roots.

With the math review out of the way, let's get into coding a solution.

Before writing the individual functions, we'll just start by figuring out the logic in the main function. Later, we will refactor it into separate functions that are testable. Here is the start of a program that can handle this problem. At this point, we are simply reading in the values for a , b , and c .

```
#include <iostream>

int main() {
    double a, b, c, x1, x2;
    std::cout << "Enter values for a, b, and c\n";
    std::cin >> a >> b >> c;
    // to be continued
    return 0;
}
```

Now, we let's write some code to test the first branch in the algorithm. We need to determine if a is 0 or not. Notice that we do not bother to implement any of the other features of the program, we are just testing if a is 0 or not and printing the appropriate response. While we wouldn't want random print statements for every detail in our final program, it is helpful for writing and testing a new program or new features.

```
#include <iostream>

int main() {
    double a, b, c, x1, x2;
    std::cout << "Enter values for a, b, and c\n";
    std::cin >> a >> b >> c;
    if (a == 0) {
        std::cout << "a is 0. this is not quadratic\n";
    }
    else {
        std::cout << "a is not 0. this is quadratic\n";
    }
    // to be continued
    return 0;
}
```

Next, let's pick a branch to continue working on. There is less to do for the branch where a is 0, so let's continue on that branch. We now need to check if b is 0. If b is 0, we are done with that branch, since there is no equation to solve.

```
#include <iostream>

int main() {
    double a, b, c, x1, x2;
    std::cout << "Enter values for a, b, and c\n";
    std::cin >> a >> b >> c;
    if (a == 0) {
        std::cout << "a is 0. this is not quadratic\n";
        if (b == 0) {
            std::cout << "b is 0. this is not a valid equation\n";
            return 0;
        }
        else {
            std::cout << "b is 0. this equation is linear.\n";
        }
    }
    else {
        std::cout << "a is not 0. this is quadratic\n";
    }
}
```

```
}  
// to be continued  
return 0;  
}
```

Once we are satisfied that our program works for testing that b is 0, we can continue to the next step. Linear equations only have 1 root, which is equal to $-c/b$. We will implement that logic, and print the result.

```
#include <iostream>  
  
int main() {  
    double a, b, c, x1, x2;  
    std::cout << "Enter values for a, b, and c\n";  
    std::cin >> a >> b >> c;  
    if (a == 0) {  
        std::cout << "a is 0. this is not quadratic\n";  
        if (b == 0) {  
            std::cout << "b is 0. this is not a valid equation\n";  
            return 0;  
        }  
        else {  
            std::cout << "b is 0. this equation is linear.\n";  
            x1 = -c / b;  
            std::cout << "there is one root. x = " << x1 << "\n";  
            return 0;  
        }  
    }  
    else {  
        std::cout << "a is not 0. this is quadratic\n";  
    }  
    // to be continued  
    return 0;  
}
```

From here, we can hand calculate some values to test for b and c and plug them in. Once we are satisfied with the results, we can move on to the next step.

Now that we are done with the $a == 0$ branch, we can handle the cases where a is not 0. The first step is to determine if the value of $b^2 - 4ac$ is greater than, less than, or equal to 0. This value is called the "discriminant" in mathematics, so I am calling it " d " in the code.

```
#include <iostream>
```

```

int main() {
    double a, b, c, x1, x2;
    std::cout << "Enter values for a, b, and c\n";
    std::cin >> a >> b >> c;
    if (a == 0) {
        std::cout << "a is 0. this is not quadratic\n";
        if (b == 0) {
            std::cout << "b is 0. this is not a valid equation\n";
            return 0;
        }
        else {
            std::cout << "b is 0. this equation is linear.\n";
            x1 = -c / b;
            std::cout << "there is one root. x = " << x1 << "\n";
            return 0;
        }
    }
    else {
        std::cout << "a is not 0. this is quadratic\n";
        double d = (b * b) - (4 * a * c);
        if (d > 0) {
            std::cout << "d is positive. we have 2 roots\n";
        }
        else if (d == 0) {
            std::cout << "d is 0. we have 1 root\n";
        }
        else {
            std::cout << "d is negative. we have 0 roots\n";
        }
    }
    // to be continued
    return 0;
}

```

We can now test with different combinations of a, b, and c to make sure our program is selecting the best path. Now, let's resolve the remaining branches. First, let's handle the case where d is positive. We need to compute the 2 roots, x1 and x2, and display them to the user. We can also resolve the d == 0 case at the same time, since we aren't strictly doing TDD.

```

#include <iostream>

int main() {
    double a, b, c, x1, x2;
    std::cout << "Enter values for a, b, and c\n";
    std::cin >> a >> b >> c;
    if (a == 0) {

```

```

std::cout << "a is 0. this is not quadratic\n";
if (b == 0) {
    std::cout << "b is 0. this is not a valid equation\n";
    return 0;
}
else {
    std::cout << "b is 0. this equation is linear.\n";
    x1 = -c / b;
    std::cout << "there is one root. x = " << x1 << "\n";
    return 0;
}
}
else {
    std::cout << "a is not 0. this is quadratic\n";
    double d = (b * b) - (4 * a * c);
    if (d > 0) {
        std::cout << "d is positive. we have 2 roots\n";
        x1 = (-b + d) / (2 * a);
        x2 = (-b - d) / (2 * a);
        std::cout << "x1 = " << x1 << "\n";
        std::cout << "x2 = " << x2 << "\n";
    }
    else if (d == 0) {
        std::cout << "d is 0. we have 1 root\n";
        x1 = -b / (2 * a);
        std::cout << "x = " << x1 << "\n";
    }
    else {
        std::cout << "d is negative. we have 0 roots\n";
    }
}
return 0;
}

```

Lab 6: Unit Testing

I have converted the above program to use functions (see below) for the various operations it performs. Your job is to define those functions, and for each function you define, write a `void` function called `test_functionName()` that uses `assert` to test the function. An example of `isQuadratic` and `test_isQuadratic` are provided. You should call these functions at the start of main before allowing the main program to execute. This way, we ensure that the functions all work as intended. Make sure your unit tests cover a wide range of possible input and output from the functions. Call the program `quad.cpp`. I would encourage you to comment out the main body of the program and focus only on the making the functions and their tests work. Then, once

all the tests work, you should be able to uncomment the body and the program should work.

Another note. Due to how floating point numbers are stored in memory, the values of two doubles that should be equal can be slightly different from each other. The `areDoublesEqual()` helper function is provided for you. You can pass it two doubles, and it will return true if the two numbers are "close enough" to be considered equal. You can use this function as the argument assert calls to compare doubles.

Example:

```
assert(areDoublesEqual(20.5, 41 / 2));
```

Here is the code for `quad.cpp`.

```
#include <iostream>
#include <cmath>
#include <cassert>

bool isQuadratic(double a);
bool isLinear(double b);
double calcLinearRoot(double b, double c);
double calcDiscriminant(double a, double b, double c);
double calcSingleRoot(double a, double b);
double calcPlusRoot(double a, double b, double c);
double calcMinusRoot(double a, double b, double c);

// test fuctions
void test_isQuadratic();
void test_isLinear();
void test_calcLinearRoot();
void test_calcDiscriminant();
void test_calcSingleRoot();
void test_calcPlusRoot();
void test_calcMinusRoot();

// helper function
bool areDoublesEqual(double testValue, double referenceValue);

int main() {
    std::cout << "Beginning unit tests...\n";
    test_isQuadratic();
    test_isLinear();
    test_calcLinearRoot();
```

```

test_calcDiscriminant();
test_calcSingleRoot();
test_calcPlusRoot();
test_calcMinusRoot();
std::cout << "All unit tests passed.\n";

double a, b, c, x1, x2;
std::cout << "Enter values for a, b, and c\n";
std::cin >> a >> b >> c;
if (isQuadratic(a) == false) {
    std::cout << "a is 0. this is not quadratic\n";
    if (isLinear(b) == false) {
        std::cout << "b is 0. this is not a valid equation\n";
        return 0;
    }
    else {
        std::cout << "b is 0. this equation is linear.\n";
        x1 = calcLinearRoot(b, c);
        std::cout << "there is one root. x = " << x1 << "\n";
        return 0;
    }
}
else {
    std::cout << "a is not 0. this is quadratic\n";
    double d = calcDiscriminant(a, b, c);
    if (d > 0) {
        std::cout << "d is positive. we have 2 roots\n";
        x1 = calcPlusRoot(a, b, c);
        x2 = calcMinusRoot(a, b, c);
        std::cout << "x1 = " << x1 << "\n";
        std::cout << "x2 = " << x2 << "\n";
    }
    else if (d == 0) {
        std::cout << "d is 0. we have 1 root\n";
        x1 = calcSingleRoot(a, b);
        std::cout << "x = " << x1 << "\n";
    }
    else {
        std::cout << "d is negative. we have 0 roots\n";
    }
}
return 0;
}

bool areDoublesEqual(double testValue, double referenceValue) {
    return std::fabs(testValue - referenceValue) < 0.0001;
}

bool isQuadratic(double a) {
    if (a == 0) return false;
    else return true;
}

```



```
}  
  
void test_isQuadratic() {  
    assert(isQuadratic(0) == false);  
    assert(isQuadratic(0) == false);  
    assert(isQuadratic(2) == true);  
    assert(isQuadratic(63) == true);  
    assert(isQuadratic(-2343) == true);  
    assert(isQuadratic(0.00000000000000000001) == true);  
}
```